

Diverse Generation from a Single Video Made Possible

Niv Haim* Ben Feinstein* Niv Granot Assaf Shocher
Shai Bagon Tali Dekel Michal Irani

Weizmann Institute of Science, Rehovot, Israel

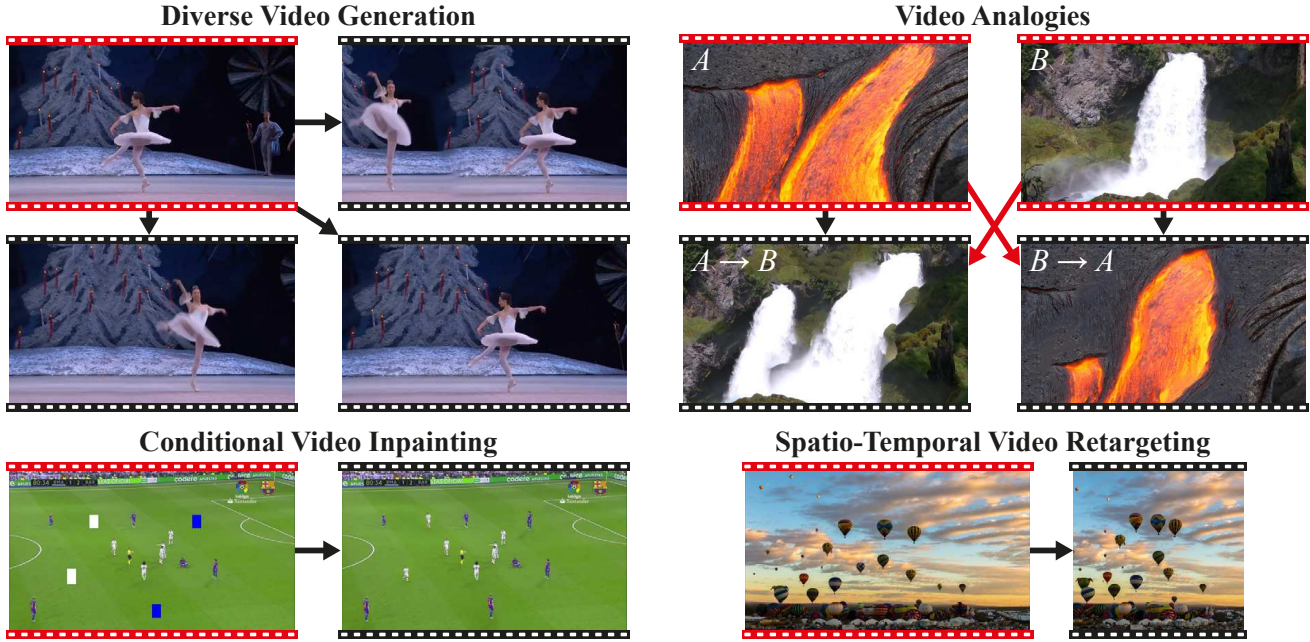


Figure 1. We adapt classical patch-based approaches as a better, much faster non-parametric alternative to single video GANs, for a variety of video generation and manipulation tasks. As we present video results, the reader is encouraged to start from the project page. Figures only present single frame examples.

Abstract

GANs are able to perform generation and manipulation tasks, trained on a single video. However, these single video GANs require unreasonable amount of time to train on a single video, rendering them almost impractical. In this paper we question the necessity of a GAN for generation from a single video, and introduce a non-parametric baseline for a variety of generation and manipulation tasks. We revive classical space-time patches-nearest-neighbors approaches and adapt them to a scalable unconditional generative model, without any learning. This simple baseline surprisingly outperforms single-video GANs in visual quality and realism (confirmed by quantitative and qualitative evaluations), and is disproportionately faster (runtime reduced from several days to seconds). Other than diverse video generation, we demonstrate other applications

using the same framework, including video analogies and spatio-temporal retargeting. Our proposed approach is easily scaled to Full-HD videos. These observations show that the classical approaches, if adapted correctly, significantly outperform heavy deep learning machinery for these tasks. This sets a new baseline for single-video generation and manipulation tasks, and no less important – makes diverse generation from a single video practically possible for the first time.

Project page: <https://nivha.github.io/vgpnn>

1. Introduction

Generation and editing of natural videos remain challenging, mainly due to their large dimensionality and the enormous space of motion they span. Most modern frameworks train generative models on a large collection of videos, producing high quality results for only a limited

*These authors contributed equally to this work.

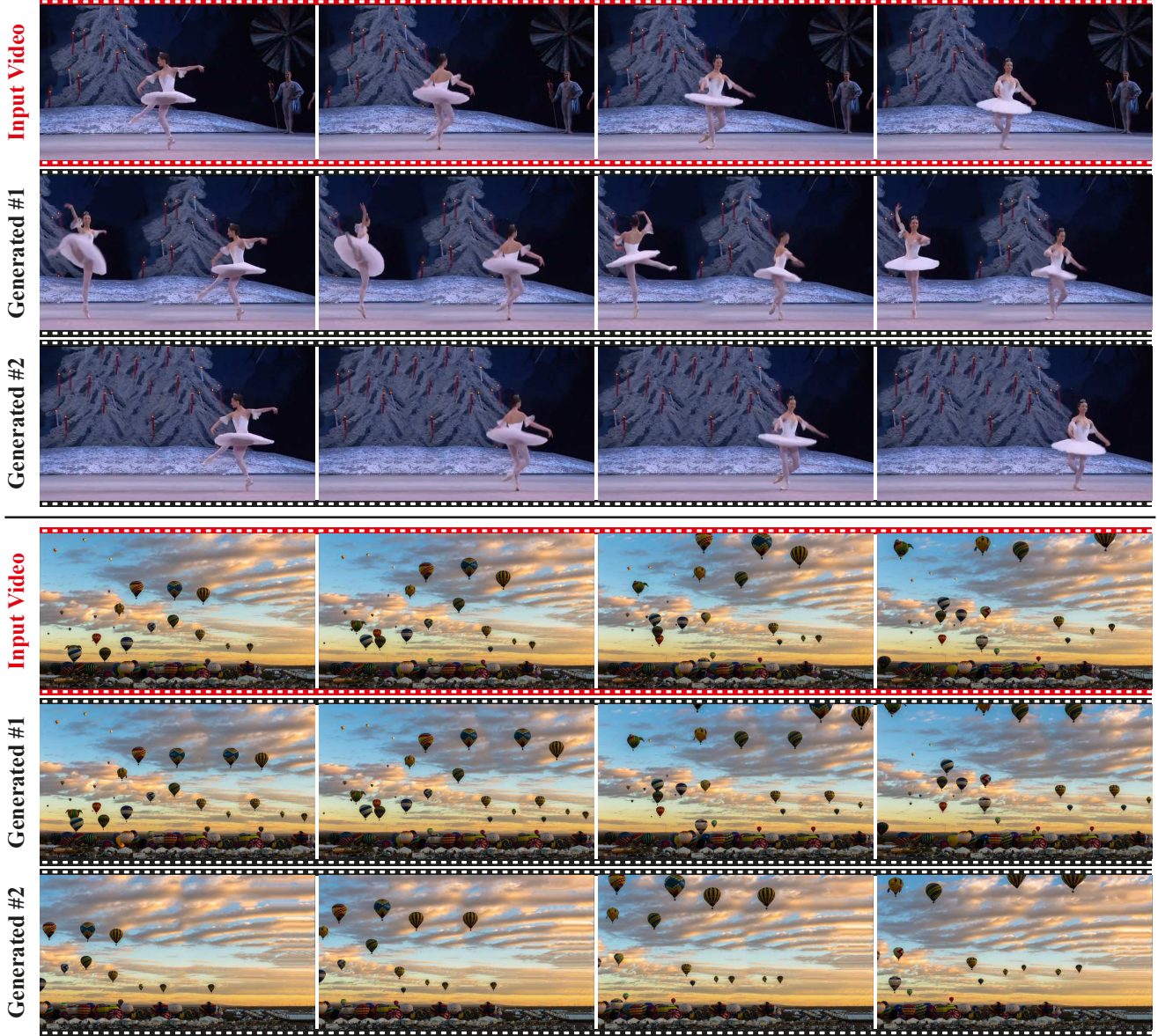


Figure 2. **Diverse Single Video Generation:** Given an input video (red), VGPNN is able to generate similarly looking videos (black) capturing both appearance of objects as well as their dynamics. Note the high quality of our generated videos. *Please watch the full resolution videos in the project page.*

class of videos. These include extensions of GANs [23] to video data [2, 36, 48, 58, 63, 67] and video-to-video translation [8, 15, 40, 64–66, 72], autoregressive sequence prediction [3, 6, 7, 17, 22, 60–62] and more. While externally-trained generative models produce impressive results, they are restricted to the types of video dynamics in their training set. On the other side of the spectrum are *single-video GANs*. These video generative models train on a *single* input video, learn its distribution of space-time patches, and are then able to generate a diversity of new videos with the same patch distribution [5, 25]. However, these take very

long time to train for each input video, making them applicable to only small spatial resolutions and to very short videos (typically, very few small frames). Furthermore, their output oftentimes shows poor visual quality and noticeable visual artifacts. These shortcomings render existing single-video GANs impractical and unscalable.

Video synthesis and manipulation of a single video sequence based on its distribution of space-time patches dates back to classical pre-deep learning methods. These classical methods demonstrated impressive results for various applications, such as video retargeting [31, 46, 55, 71], video

completion [27, 39, 70], video texture synthesis [14, 21, 28, 32–34] and more. With the rise of deep-learning, these methods gradually, perhaps unjustifiably, became less popular. Recently, Granot et al. [24] revived classical patch-based approaches for image synthesis, and was shown to significantly outperform *single-image* GANs in both runtime and visual quality.

In light of the above-mentioned deficiencies of single-video GANs, and inspired by [24], we propose VGPNN (*Video Generative Patch Nearest Neighbors*), a fast and practical method for video generation and manipulation from a single video. In order to handle the huge amounts of space-time patches in a single video sequence, we use and extend classical fast approximate nearest neighbor search methods [9]. We also employ robust optical-flow-like descriptors, which allow transferring highly different dynamics and motions from one video to another. Finally, by adding stochastic noise to the process, our approach can generate a large diversity of random different video outputs from a single input video in an unconditional manner.

Like single-video GANs, our approach enables the diverse and random generation of videos. However, in contrast to existing single-video GANs, VGPNN can generate *high resolution* videos, while reducing runtime by many orders of magnitude, thus making diverse unconditional video generation from a single video realistically possible for the first time.

Our contributions are as follows:

- We observe that space-time patch nearest-neighbor approaches, when posed as an unconditional generative model, outperform single-video GANs by a large margin, both in runtime and in quality.
- We provide a new baseline for comparing single-video generative models. Our approach is practical to run and compare against (code will be released), and scalable also to high resolution videos (spatial or temporal).
- We demonstrate a variety of video synthesis tasks, all within a single unified framework. These include: diverse unconditional video generation, video analogies, sketch-to-video, spatio-temporal video retargeting and conditional video inpainting.

2. Related work

Classical video generation methods, many of whom inspired by similar *image* methods [19, 20, 68], include video texture synthesis [32–34], MRF-based controllable synthesis [51], flow-guided synthesis [14, 28, 34, 41, 49, 50] and more (see surveys by [11, 69]). While some used a generative model to model patch distribution, none considered unconditional generation of natural videos, beyond dynamic textures.

PatchMatch by Barnes et al. [9] is a fast method for finding an approximate nearest-neighbor field (NNF). On natu-

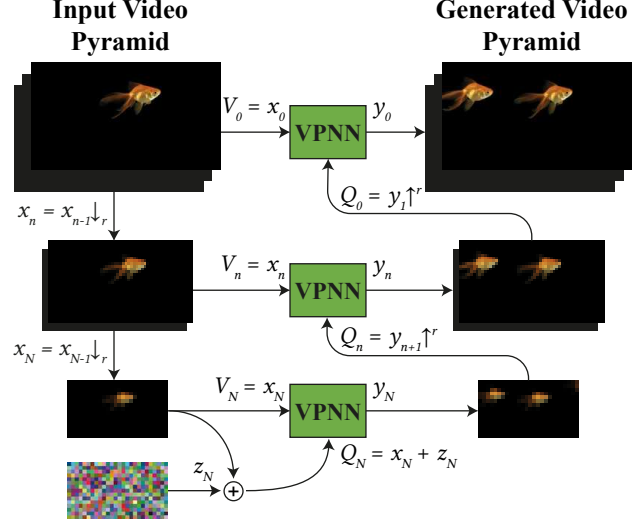


Figure 3. **VGPNN Architecture:** Given a single input video x_0 , a spatio-temporal pyramid is constructed and an output video y_0 is generated coarse-to-fine. At each scale, VPNN module (see Fig. 4) is applied to transfer an initial guess Q_n to the output y_n which shares the same space-time patch distribution as the input x_n . At the coarsest scale, noise is injected to induce spatial and temporal randomness.

ral images/videos, the algorithm converges very quickly to a good approximated solution. PatchMatch was also extended for k-nearest neighbors search [10], faster search [11] and being differentially learnable [18]. The main editing tool shown in [9] is image summarization using bidirectional similarity (BDS). Introduced by Simakov et al. [55], BDS ensures both visual “completeness” and “coherence” of the visual summary (coherence is obtained when the generated output contains only patches from the input, while completeness is obtained when all the patches in the input can be found in the generated output). [24] proposed a normalized score to encourage completeness by globally re-weighting the similarity score between patches. Motivated by this similarity score, we utilize PatchMatch to efficiently incorporate global patch-specific information.

3. Method

Our goal is to generate diverse video samples based on a single input video. We want our model to operate on natural input videos that can vary in their appearance and dynamics. In order to capture both spatial and temporal information of a single video, we start by building a spatio-temporal pyramid and operate coarse-to-fine to capture the internal statistics of the input video at multiple scales (Fig. 3). This multi-scale approach is extensively used in classical image synthesis methods as well as in modern GANs [e.g., 25, 30, 52]). At each scale we employ a Video-Patch-Nearest-Neighbor module or VPNN (VGPNN is in fact a

sequence of VPNN layers). The inputs to each layer depend on the application, where we first focus on our main application of diverse video generation (see Sec. 5 for the specific details of the other applications).

Multi-scale approach (Fig.3): Given an input video x , we construct a spatio-temporal pyramid $\{x_0 \dots, x_N\}$, where $x_0 = x$, and $x_n = x_{n-1} \downarrow_r$ is a bicubically down-scaled version of x_{n-1} by factor r ($r = (r_H, r_W, r_T)$), where $r_H = r_W$ are the spatial factors and r_T is the temporal factor, which can be different).

At the coarsest scale, the input to the first VPNN layer is an initial coarse guess of the output video. This is created by adding random Gaussian noise z_N to x_N . The noise z_N promotes high diversity in the generated output samples from the single input. The global structure (e.g., a head is above the body) and global motion (e.g., humans walk forward), is prompted by x_N , where such structure and motion can be captured by *small space-time* patches. The coarsest-scale output y_N is generated by replacing each space-time patch of the initial coarse guess ($x_N + z_N$) with its nearest neighbor patch from the corresponding coarse input x_N . The resulting patches are then folded to a video, by choosing at each space-time position the median of all suggestions from neighboring patches.

At each subsequent scale, the input to the VPNN layer is the bicubically-upscaled output of the previous layer ($y_{n+1} \uparrow^r$). The output y_n is then generated by replacing each space-time patch with its nearest neighbor patch from the corresponding input x_n (using the same patch-size as before, now capturing finer details). This way, the output y_n in each scale is similar in structure and in motion to the initial guess, but contains the same space-time patch statistics of the corresponding input x_n . Finally, the resulting patches are folded to a video.

To further improve the quality and sharpness of the generated output at each pyramid scale (y_n), we iterate several times through the current scale VPNN layer, each time using the current output y_n as input to the next iteration (similar to the EM-like approach employed in many patch-based works [e.g., 9, 24, 55, 70]).

QKV scheme (Fig. 4): Similar to [24], we adopt a QKV scheme (query, key and value, respectively; as is common in the attention mechanism [59] nomenclature) for VPNN. Instead of comparing two patches by only using their RGB values, in several cases it is necessary to compare patches in another search space. For example, using the aforementioned notations, we denote $V = x_n$ (the corresponding level from the pyramid of the original video) and $Q = y_{n+1} \uparrow$ (the upscaled output of previous layer). Note that since Q is an *upscaled* version of previous output, its patches are blurry. Seeking their nearest neighbors

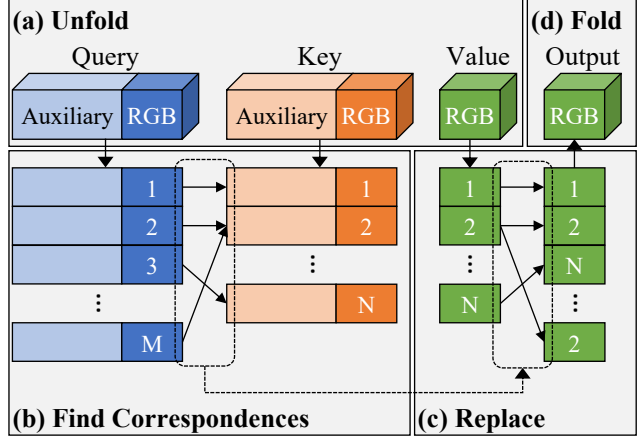


Figure 4. **VPNN module** gets as input RGB videos query, key and value (QKV respectively). Q and K can be concatenated to additional auxiliary channels. It outputs an RGB video. (a) Inputs are unfolded to patches (each position now holds a concatenation of neighboring positions); (b) Each patch in Q finds its nearest neighbor patch in K. This is achieved by solving the NNF using PatchMatch [9]; (c) Each patch in Q is replaced with a patch from V, according to the correspondences found in stage (b); (d) Resulting patches are folded back to an RGB video output.

in V (whose patches are sharp) often results in improper matches. This is mitigated by setting $K = x_{n+1} \uparrow^r$ (in the first iteration of each scale of VGPNN), which has a similar degree of blur/degradation as Q . After finding its match in K_j , each patch Q_i is then replaced with a patch V_j (where i, j are spatio-temporal positions. Also note that K and V are of the same shape).

The QKV scheme is especially important in our video analogies application where it is used to include additional temporal information in the queries and the keys. We discuss it in detail in Sec. 5.1.

Completeness score: In the applications of video analogies, spatio-temporal video retargeting and conditional video inpainting we use the normalized similarity score [24] that encourages visual completeness. The score between a query patch Q_i and a key patch K_j is defined as:

$$S(Q_i, K_j) := \frac{1}{\alpha + \min_{\ell} D(Q_{\ell}, K_j)} D(Q_i, K_j) , \quad (1)$$

where $D = MSE$, and α controls the degree of completeness (smaller α encourages more completeness). S is essentially a weighted version of D , whose weights depend globally on K and Q .

Finding Correspondences: We find the nearest neighbors between Q and K (Fig. 4b) using PatchMatch (Barnes et al. [9]). To cope with the completeness score, we apply

PatchMatch twice. First we find a “rareness” score for the keys - for each *key* we find its closest *query*. Then, for each *query* we find its closest *key* while factoring in the rareness of the keys as weights in the PatchMatch search. Namely, we solve for:

$$\text{NNF}(\mathbf{p}) = \arg \min_{\mathbf{v}} W(\mathbf{p} + \mathbf{v}) \cdot D(Q(\mathbf{p}), K(\mathbf{p} + \mathbf{v})) \quad (2)$$

Where D is a distance function, W are per-patch weights, $\mathbf{p} = (t, x, y)$ a position in Q and $\mathbf{v} = (t', x', y')$ are possible NNF candidates (such as the NNF at the current position $\text{NNF}(t, x, y)$ or at a neighbor position $\text{NNF}(t, x - 1, y)$ in the propagation step).

This requires a slight modification of PatchMatch to support per-key weights. This additional support makes it possible to approximately solve Eq. 1 with two passes of PatchMatch. Even though this gives an approximation of Eq. 1, we do not suffer loss in quality or lack of completeness, as apparent from our results.

The algorithm is implemented on GPU using PyTorch [42], with time complexity $O(n \times d)$ and $O(n)$ additional memory (where n is the video size and d is the patch size; also see Fig. 6).

Temporal Diversity and Consistency: To enhance the temporal diversity of our samples we set the temporal dimension of the output to be slightly smaller than that of the input video. Thus, motions in different spatial positions in the generated output are taken from different temporal positions in the input video, increasing the overall temporal diversity (see for example the generated dancers in Fig. 2 that are not synced). We also found that the temporal consistency is best preserved in the generated output when the initial noise z_N is randomized for each spatial position, but is the same (replicated) in the temporal dimension.

4. Experimental Results

In this section we evaluate and compare the performance of our main application – diverse video generation from a single input video. Figs. 1 and 2 illustrate diverse videos generated from a single input video, all sharing the same space-time patch distribution. The diversity is both spatially (e.g., number of dancers and their positions are different from the input video) and temporally (generated dancers are not synced). Please refer to the supplementary material to view the full resolution videos and many more examples.

Comparison to other methods for video generation from a single video: We compare our method to recently published methods for diverse video generation from single video: HP-VAE-GAN [25] and SinGAN-GIF [5]. We show that our results are both qualitatively and quantitatively superior while reducing the runtime by a factor of 3×10^4

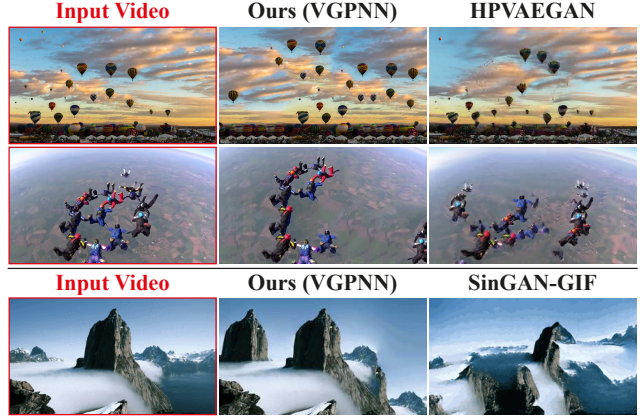


Figure 5. **Comparing Visual Quality** between our generated frames and those of HP-VAE-GAN [25] and SinGAN-GIF [5] (please **zoom in** on the frames). Note that our generated frames are sharper and also exhibit more coherent and plausible arrangements of the scene. For details see Section 4. Please find the full videos and more comparisons in the supplementary material.

(from 8 days training on one video to 18 seconds for new generated video). Since SinGAN-GIF did not make their code available, and the training time of HP-VAE-GAN for a single video (13 frames of size 144×256) is roughly 8 days, we are only able to compare to the videos published by these methods. “HP-VAE-GAN dataset” comprises of 10 input videos with 13 frames each, and of spatial resolution of 144×256 pixels. “SinGAN-GIF dataset” has 5 input videos with maximal resolution of 168×298 pixels and 8-16 frames.

Qualitative comparison: In Fig. 5 we show a side-by-side comparison of representative generated frames of our method to frames generated by HP-VAE-GAN [25] and SinGAN-GIF [5]. Note that while [5, 25] are limited to generated outputs of small resolution (144×256), we can generate outputs in the same resolution of the input video (full-HD 1280×1920 , shown in the figure). The full videos (as well as a comparison to our generated outputs of similar low resolution) can be viewed in the supplementary material. As can be seen, our generated samples (in low and high resolution) are more spatially and temporally coherent, as well as having higher visual quality. It is evident that generating videos using the space-time patches of the original input video, rather than regressing output RGB values, gives rise to high quality outputs.

Quantitative comparison: In Table 1 we report the Single-Video-FID (SVFID) [5] of our generated samples, compared to those generated by HP-VAE-GAN [25] and SinGAN-GIF [5]*. SVFID was proposed by [25] to mea-

*All quantitative comparisons were done on generated samples of the same resolution and video length as that of the other method.

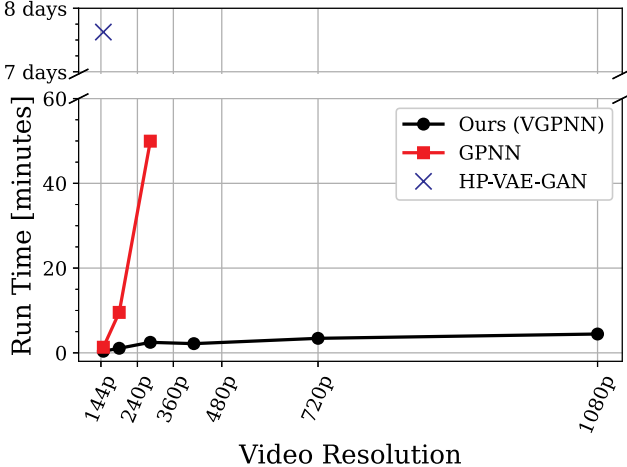


Figure 6. **Comparing Generation Runtime** between our approach (VGPNN), a naïve extension of GPNN [24] from 2D to 3D and HP-VAE-GAN [25]. We compared the generation time of 13-frames videos with different spatial resolutions (X-axis). All videos have 16:9 aspect ratio (e.g., 144p is 144x256 and 1080p is 1080x1920 – full-HD). See Section 4 for details.

sure the patch statistics similarity between the input video and a generated video. It computes the Fréchet distance between the statistics of the input video and the generated video using pre-computed C3D [57] features (Lower SVFID is better). As can be seen in Table 1, our generated samples bear more substantial similarity to the input videos (indicated by lower SVFID). [52] proposed a diversity index to make sure that generated outputs are indeed different (and not simply “copying” the input). We adapt the index for videos. The index is zero if all generated outputs are the same, and higher otherwise. While our and HP-VAE-GAN generated samples have similar index (0.45/0.41 respectively), those of SinGAN-GIF have higher index (0.86 vs. our 0.6). Such high diversity is not an advantage, when paired with SVFID about twice worse than ours. It stems from low quality appearance with out-of-distribution patches. All inputs and generated videos can be found in the supplementary material.

User study: We conducted a user study evaluation using Amazon Mechanical Turk (AMT). For each dataset, 100 subjects were shown multiple pairs of videos, each consisting of a video generated by our method, and a video generated by the other method (both were generated from the same input video). The subjects were asked to judge which sample is better in terms of sharpness, natural look and coherence. In Table 1 we report the percentage of users who favored our method over the other. Compared to videos generated from HP-VAE-GAN dataset, there is a clear preference in favor of our patch-based method. The results on

Method	SVFID [25] ↓	Head-on comparison (User study) [%] ↑	Runtime ↓
HP-VAE-GAN [25]	0.0081	67.84 ±1.77	7.625 days
VGPNN (Ours)	0.0072		18 secs
SinGAN-GIF [5]	0.0119	50.57 ± 3.27	Unpublished
VGPNN (Ours)	0.0058		10 secs

Table 1. **Quantitative Evaluation:** A comparison of our generated video samples to that of HP-VAE-GAN [25] and SinGAN-GIF [5], conducted on input videos provided in their papers. Our diverse samples have more resemblance to the input videos (indicated by lower SVFID). In a user study, users scored in favor of our method (see Section 4 for details).

the SinGAN-GIF dataset are not that clear-cut, this might be due to the somewhat restricted nature of the videos in that particular dataset (as mentioned above, it was not possible to check SinGAN-GIF on other samples, since the authors did not publish their code, nor stated the amount of time it took to generate their samples).

Reducing running times: In Fig. 6 we show a comparison of the runtime taken to generate random video samples using our method, compared to a naïve extension of GPNN [24] (from 2D to 3D patches) and compared to the training time of HP-VAE-GAN [25]. As discussed in Sec. 3, the use of efficient PatchMatch algorithm for nearest neighbors search, as opposed to the exhaustive search done in GPNN, dramatically reduces both run time and memory footprint used for video generation, making it possible to generate high-resolution videos (including Full-HD 1080p). All experiments were conducted on Quadro RTX 8000 GPU.

5. Applications

Other than unconditional diverse generation, we demonstrate the utility of the proposed unified framework on several other video manipulation applications.

5.1. Video analogies

Inspired by *Image Analogies* [12, 26, 38] we propose *Video Analogies* where we generate a new video whose spatio-temporal layout is taken from a content video C , and overall appearance and dynamics are taken from a style video S (see Fig. 7).

Our goal is to find a mapping of dynamic elements (patches) between the two videos, which can be very different in their appearance (RGB space). This is achieved by using the magnitude of the optical flow (extracted via RAFT [56]), quantized into few bins (using k-means)[†]. We

[†]Each cluster has an integer cluster index. We divide each index by the total number of clusters/bins to be in $[0, 1]$

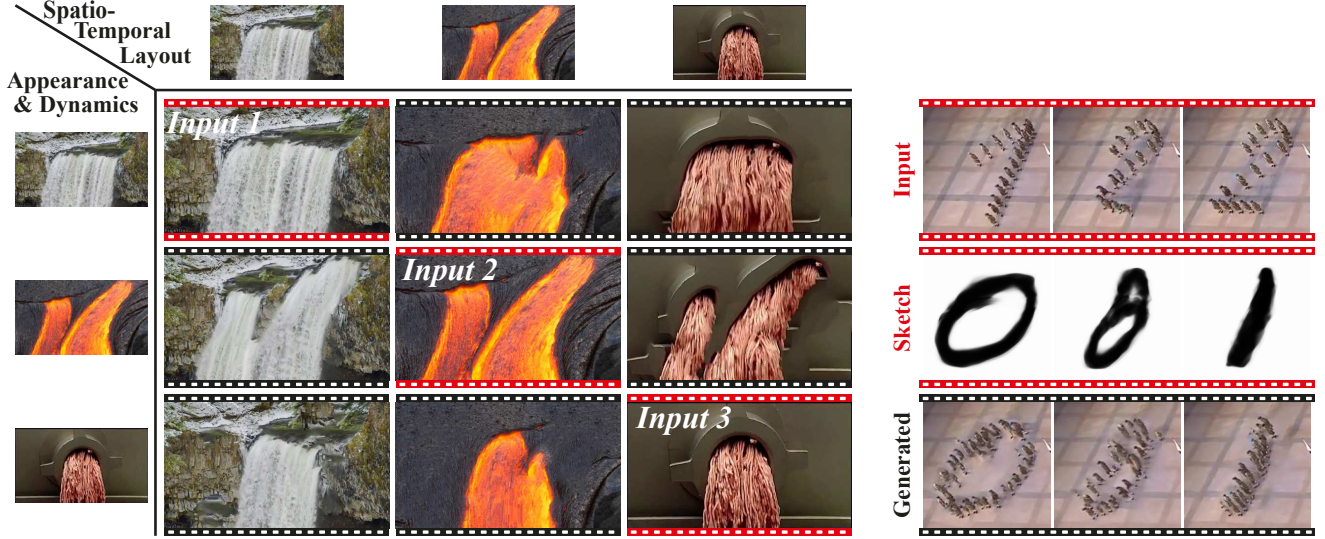


Figure 7. **Video Analogies:** *Left:* an example of video analogies between all pairs of three input videos (red). Each generated video (black) takes the spatio-temporal layout from the input video in its row, and the appearance and dynamics of the input video from its column. *Right:* an example of sketch-to-video – the generated video (bottom) takes its spatio-temporal layout from the sketch video of morphed MNIST digits (middle) and its appearance and dynamics from the input video of parading soldiers (top). *Please find full videos and additional examples in the supplementary material.*

term this the *dynamic structure* of the video. By concatenating the dynamic structure to the RGB values of the video (along the channels axis), each patch can now be compared using its RGB values and its dynamic values. This provides a good mapping between the dynamic elements of the two input videos.

Given the dynamic structures of both input videos $\text{dyn}(C)$, $\text{dyn}(S)$, we compute their spatio-temporal pyramids and that of the style video S . The output video is generated by setting Q , K , V at each scale as follows:

Scale	Q	K	V
N (coarsest)	$\text{dyn}(C)_N$	$\text{dyn}(S)_N$	S_N
n (any other)	$\text{dyn}(C)_n \parallel Q_{n+1} \uparrow$	$\text{dyn}(S)_n \parallel S_n$	S_n

where $\parallel$ denotes concatenation along the channels axis, and n denote the current scale in the pyramid. Note that in the coarsest scale, the two videos are only compared by their dynamic structure. In finer scales, the dynamic structure of C (the content video) is used to “guide” the output to the desired spatio-temporal layout.

In Fig. 1 we show a snapshot of the analogies between a waterfall and a lava stream, and in Fig. 7 we show snapshots of the analogies of all possible pairs between three videos (the lava stream, a waterfall and a meat grinder). *The full videos are in the supplementary material.*

We can use the above mentioned mechanism for “sketch-to-video” transfer, where the dynamic structure is given by a sketch video instead of an actual video. See Fig. 7

for a few snapshots of transferring the motion of morphed MNIST [35] digits to a video of marching soldiers, and *please see the full videos and many more results in the supplementary material.*

Related to us are works for video style transfer from a style image [e.g. 13, 16, 29, 47], general video-to-video translation trained on large datasets or conditioned on human poses or keypoint detection [e.g. 40, 64–66] or human motion transfer methods [e.g. 1, 15, 37, 44, 53, 54, 72] that involve some kind of human model knowledge. Flow-based appearance transfer of fluids has been studied by [14, 28, 34, 41, 49, 50]. Most similar to us is [28] that uses a patch nearest neighbor approach to transfer the appearance of a fluid exemplar (a still image) into a video given a human annotated flow+alpha mask. Our method differs in how we model the flow guidance and in the mapping we have between two flows of two videos (instead of a still image exemplar).

Also similar to us is Recycle-GAN by Bansal et al. [8] that pose unsupervised video-to-video translation as a domain transfer problem (each video is a domain). They train convolutional encoders to map between the two videos using adversarial loss with cyclic constraints. We provide a comparison to [8] in the supplementary material. As can be seen for the sketch to videos examples, RecycleGAN’s outputs fail to capture the finer motions of the soldiers, and in the fluid-like examples it quickly converges to the style video. On top of that, the overall visual quality is diminished due to the model being parametric.

5.2. Spatial Retargeting

The goal of video retargeting is to change the dimensions of a video without distorting its visual contents (e.g., fit a portrait video to a wide screen display). It can be performed in a very similar manner to our video generation described in Sec.3. Given a target shape, we first resize (bicubically) the input video to the target shape, then compute two pyramids (for the input and resized videos) with the same depth and downscale factor. The initial guess at the coarsest scale Q_N would be the coarsest scale of the resized pyramid (without any additional noise). We then compute the rest of the output video in the same manner as in Sec.3. Note that at each scale, V_n are unchanged, hence no distortion is introduced to the patches reconstructing the retargeted video.

As can be seen in Fig. 1 and in the supplementary material, the results preserve the original size and aspect ratio of objects from the input videos while keeping the overall appearance coherent even though the aspect ratio is significantly altered. The dynamics and motions in the videos are also preserved. For instance, the balloons are not “squashed” but rather packed more compactly in the sky and more members were added to the choir instead of stretching them. Nevertheless, the motion of the balloons or the sway of the choir members are preserved. Other classical works for video retargeting, such as [31, 46, 55, 71] did not make their implementation available, therefore we were unable to provide a comparison.

5.3. Temporal Retargeting

Similar to spatial retargeting, one can generate a realistic video with a different *temporal* length. One possible use is generating a shorter summary of the video. While most deep video summarization techniques are achieved by selecting a subset of frames (see survey [4]), classical methods have demonstrated summaries that consist of *novel frames* in which dynamics that are originally sequential can be parallelized or vice versa [43, 55]. By applying the retargeting approach to the temporal dimension, VGPNN generates summaries with novel frames. The *temporal retargeting section in the supplementary material* shows several examples. For example, in the dog training summarized video, the trainer and dog turn around simultaneously as opposed to sequentially in the original video. Moreover, we can, in a similar manner, extend the temporal duration of a video creating longer dynamics while preserving the speed of the individual actions. In the ballet dancer video for example, the choreography is longer, but the pace of the dance motions remains the same.

5.4. Video conditional inpainting

In this task we are given an input video with some occluded space-time volume, where the missing parts should



Figure 8. **Conditional Video Inpainting.** Colored occluded masks in the input video (red) are completed, conditioned on the color cue. The completion persists through the video dynamics. Please see full videos in the supplementary material.

be completed based on crude color cues placed by the user in the occluded space (similar to conditional image inpainting [24]). Here we set the number of levels in the pyramid such that the occluded part in the coarsest scale is roughly in the size of a single patch. The masked part is then coherently reconstructed using other space-time patches of similar colors to that of the cue. In finer scales, details and dynamic elements are added correctly. Fig. 8 shows how different cues are completed with different elements from the non-occluded parts. See for instance, how a blue cue will be replaced by a player from Barcelona while a white cue by a player from Real Madrid. See more examples in the supplementary material.

6. Limitations

Generation of local patches, as in VGPNN but also as in single video GANs, lacks global geometric consistency. This is apparent when scenes with significant depth variations are introduced with large camera motion. While each frame is plausible, different patches are not being transformed consistently, resulting in non-rigid deformations to entities that are realistically rigid. See the generated videos of mountains in the supplementary.

7. Conclusion

We demonstrated that random diverse video generation from a single video can be efficiently done by simple patch-based methods. We also demonstrated how small modifications to our framework give rise to other tasks such as video analogies and spatio-temporal retargeting. We showed that our non-parametric approach outperforms existing single-video GANs in the visual quality of the generated outputs, while being orders of magnitude faster. The low run time required for generating videos using our approach makes it a good baseline for future works in the field.

Acknowledgements

This project received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme (grant agreement No 788535), from the D. Dan and Betty Kahn Foundation, and from the Israel Science Foundation (grant 2303/20). Dr. Bagon is a Robin Chemers Neustein AI Fellow.

References

- [1] K. Aberman, Y. Weng, D. Lischinski, D. Cohen-Or, and B. Chen. Unpaired motion style transfer from video to animation. *ACM Transactions on Graphics (TOG)*, 39(4):64–1, 2020. 7
- [2] S. Aigner and M. Körner. Futuregan: Anticipating the future frames of video sequences using spatio-temporal 3d convolutions in progressively growing gans. *arXiv preprint arXiv:1810.01325*, 2018. 2
- [3] E. Aksan and O. Hilliges. Stcn: Stochastic temporal convolutional networks. *arXiv preprint arXiv:1902.06568*, 2019. 2
- [4] E. Apostolidis, E. Adamantidou, A. I. Metsai, V. Mezaris, and I. Patras. Video summarization using deep neural networks: A survey. *arXiv preprint arXiv:2101.06072*, 2021. 8
- [5] R. Arora and Y. J. Lee. Singan-gif: Learning a generative video model from a single gif. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1310–1319, 2021. 2, 5, 6
- [6] M. Babaeizadeh, C. Finn, D. Erhan, R. H. Campbell, and S. Levine. Stochastic variational video prediction. *arXiv preprint arXiv:1710.11252*, 2017. 2
- [7] N. Ballas, L. Yao, C. Pal, and A. Courville. Delving deeper into convolutional networks for learning video representations. *arXiv preprint arXiv:1511.06432*, 2015. 2
- [8] A. Bansal, S. Ma, D. Ramanan, and Y. Sheikh. Recycle-gan: Unsupervised video retargeting. In *Proceedings of the European conference on computer vision (ECCV)*, pages 119–135, 2018. 2, 7
- [9] C. Barnes, E. Shechtman, A. Finkelstein, and D. B. Goldman. Patchmatch: A randomized correspondence algorithm for structural image editing. *ACM Trans. Graph.*, 28(3):24, 2009. 3, 4, 12
- [10] C. Barnes, E. Shechtman, D. B. Goldman, and A. Finkelstein. The generalized patchmatch correspondence algorithm. In *European Conference on Computer Vision*, pages 29–43. Springer, 2010. 3
- [11] C. Barnes, F.-L. Zhang, L. Lou, X. Wu, and S.-M. Hu. Patchtable: Efficient patch queries for large datasets and applications. *ACM Transactions on Graphics (ToG)*, 34(4):1–10, 2015. 3
- [12] S. Benaïm, R. Mokady, A. Bermano, and L. Wolf. Structural analogy from a single image pair. In *Computer Graphics Forum*, volume 40, pages 249–265. Wiley Online Library, 2021. 6
- [13] P. Bénard, F. Cole, M. Kass, I. Mordatch, J. Hegarty, M. S. Senn, K. Fleischer, D. Pesare, and K. Breeden. Stylizing animation by example. *ACM Transactions on Graphics (TOG)*, 32(4):1–12, 2013. 7
- [14] K. S. Bhat, S. M. Seitz, J. K. Hodgins, and P. K. Khosla. Flow-based video synthesis and editing. In *ACM SIGGRAPH 2004 Papers*, pages 360–363. 2004. 3, 7
- [15] C. Chan, S. Ginosar, T. Zhou, and A. A. Efros. Everybody dance now. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5933–5942, 2019. 2, 7
- [16] D. Chen, J. Liao, L. Yuan, N. Yu, and G. Hua. Coherent online video style transfer. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1105–1114, 2017. 7
- [17] E. Denton and R. Fergus. Stochastic video generation with a learned prior. In *International Conference on Machine Learning*, pages 1174–1183. PMLR, 2018. 2
- [18] S. Duggal, S. Wang, W.-C. Ma, R. Hu, and R. Urtasun. Deep-pruner: Learning efficient stereo matching via differentiable patchmatch. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4384–4393, 2019. 3
- [19] A. A. Efros and W. T. Freeman. Image quilting for texture synthesis and transfer. In *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, pages 341–346, 2001. 3
- [20] A. A. Efros and T. K. Leung. Texture synthesis by non-parametric sampling. In *Proceedings of the seventh IEEE international conference on computer vision*, volume 2, pages 1033–1038. IEEE, 1999. 3
- [21] J. Fišer, O. Jamriška, M. Lukáč, E. Shechtman, P. Asente, J. Lu, and D. Šykora. Stylit: illumination-guided example-based stylization of 3d renderings. *ACM Transactions on Graphics (TOG)*, 35(4):1–11, 2016. 3
- [22] J.-Y. Franceschi, E. Delasalles, M. Chen, S. Lamprier, and P. Gallinari. Stochastic latent residual video prediction. In *International Conference on Machine Learning*, pages 3233–3246. PMLR, 2020. 2
- [23] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial networks. *arXiv preprint arXiv:1406.2661*, 2014. 2
- [24] N. Granot, B. Feinstein, A. Shocher, S. Bagon, and M. Irani. Drop the gan: In defense of patches nearest neighbors as single image generative models. *arXiv preprint arXiv:2103.15545*, 2021. 3, 4, 6, 8, 12

- [25] S. Gur, S. Benaim, and L. Wolf. Hierarchical patch vae-gan: Generating diverse videos from a single sample. *arXiv preprint arXiv:2006.12226*, 2020. 2, 3, 5, 6
- [26] A. Hertzmann, C. E. Jacobs, N. Oliver, B. Curless, and D. H. Salesin. Image analogies. In *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, pages 327–340, 2001. 6
- [27] J.-B. Huang, S. B. Kang, N. Ahuja, and J. Kopf. Temporally coherent completion of dynamic video. *ACM Transactions on Graphics (TOG)*, 35(6):1–11, 2016. 3
- [28] O. Jamriška, J. Fišer, P. Asente, J. Lu, E. Shechtman, and D. Šykora. Lazyfluids: appearance transfer for fluid animations. *ACM Transactions on Graphics (TOG)*, 34(4):1–10, 2015. 3, 7
- [29] O. Jamriška, Š. Sochorová, O. Texler, M. Lukáč, J. Fišer, J. Lu, E. Shechtman, and D. Šykora. Stylizing video by example. *ACM Transactions on Graphics (TOG)*, 38(4):1–11, 2019. 7
- [30] T. Karras, T. Aila, S. Laine, and J. Lehtinen. Progressive growing of gans for improved quality, stability, and variation. *arXiv preprint arXiv:1710.10196*, 2017. 3
- [31] P. Krähenbühl, M. Lang, A. Hornung, and M. Gross. A system for retargeting of streaming video. In *ACM SIGGRAPH Asia 2009 papers*, 2009. 2, 8
- [32] V. Kwatra, A. Schödl, I. Essa, G. Turk, and A. Bobick. Graphcut textures: Image and video synthesis using graph cuts. *Acm transactions on graphics (tog)*, 22(3):277–286, 2003. 3
- [33] V. Kwatra, I. Essa, A. Bobick, and N. Kwatra. Texture optimization for example-based synthesis. In *ACM SIGGRAPH 2005 Papers*, pages 795–802. 2005.
- [34] V. Kwatra, D. Adalsteinsson, T. Kim, N. Kwatra, M. Carlson, and M. Lin. Texturing fluids. *IEEE transactions on visualization and computer graphics*, 13(5):939–952, 2007. 3, 7
- [35] Y. LeCun. The mnist database of handwritten digits. <http://yann.lecun.com/exdb/mnist/>, 1998. 7
- [36] A. X. Lee, R. Zhang, F. Ebert, P. Abbeel, C. Finn, and S. Levine. Stochastic adversarial video prediction. *arXiv preprint arXiv:1804.01523*, 2018. 2
- [37] J. Lee, D. Ramanan, and R. Girdhar. Metapix: Few-shot video retargeting. *arXiv preprint arXiv:1910.04742*, 2019. 7
- [38] J. Liao, Y. Yao, L. Yuan, G. Hua, and S. B. Kang. Visual attribute transfer through deep image analogy. *arXiv preprint arXiv:1705.01088*, 2017. 6
- [39] M. Liu, S. Chen, J. Liu, and X. Tang. Video completion via motion guided spatial-temporal global optimization. In *Proceedings of the 17th ACM international conference on Multimedia*, pages 537–540, 2009. 3
- [40] A. Mallya, T.-C. Wang, K. Sapra, and M.-Y. Liu. World-consistent video-to-video synthesis. *arXiv preprint arXiv:2007.08509*, 2020. 2, 7
- [41] M. Okabe, K. Anjyo, T. Igarashi, and H.-P. Seidel. Animating pictures of fluid using video examples. In *Computer Graphics Forum*, volume 28, pages 677–686. Wiley Online Library, 2009. 3, 7
- [42] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. URL <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>. 5
- [43] A. Rav-Acha, Y. Pritch, and S. Peleg. Making a long video short: Dynamic video synopsis. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, volume 1, pages 435–441. IEEE, 2006. 8
- [44] J. Ren, M. Chai, S. Tulyakov, C. Fang, X. Shen, and J. Yang. Human motion transfer from poses in the wild. In *European Conference on Computer Vision*, pages 262–279. Springer, 2020. 7
- [45] G. Rong and T.-S. Tan. Jump flooding in gpu with applications to voronoi diagram and distance transform. In *Proceedings of the 2006 symposium on Interactive 3D graphics and games*, pages 109–116, 2006. 12
- [46] M. Rubinstein, A. Shamir, and S. Avidan. Improved seam carving for video retargeting. *ACM transactions on graphics (TOG)*, 27(3):1–9, 2008. 2, 8
- [47] M. Ruder, A. Dosovitskiy, and T. Brox. Artistic style transfer for videos. In *German conference on pattern recognition*, pages 26–36. Springer, 2016. 7
- [48] M. Saito, E. Matsumoto, and S. Saito. Temporal generative adversarial nets with singular value clipping. In *Proceedings of the IEEE international conference on computer vision*, pages 2830–2839, 2017. 2
- [49] S. Sato, Y. Dobashi, T. Kim, and T. Nishita. Example-based turbulence style transfer. *ACM Transactions on Graphics (TOG)*, 37(4):1–9, 2018. 3, 7
- [50] S. Sato, Y. Dobashi, and T. Nishita. Editing fluid animation using flow interpolation. *ACM Transactions on Graphics (TOG)*, 37(5):1–12, 2018. 3, 7
- [51] A. Schödl, R. Szeliski, D. H. Salesin, and I. Essa. Video textures. In *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, pages 489–498, 2000. 3

- [52] T. R. Shaham, T. Dekel, and T. Michaeli. Singan: Learning a generative model from a single natural image. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4570–4580, 2019. 3, 6, 13
- [53] A. Siarohin, S. Lathuilière, S. Tulyakov, E. Ricci, and N. Sebe. Animating arbitrary objects via deep motion transfer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2377–2386, 2019. 7
- [54] A. Siarohin, S. Lathuilière, S. Tulyakov, E. Ricci, and N. Sebe. First order motion model for image animation. *Advances in Neural Information Processing Systems*, 32:7137–7147, 2019. 7
- [55] D. Simakov, Y. Caspi, E. Shechtman, and M. Irani. Summarizing visual data using bidirectional similarity. In *2008 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8. IEEE, 2008. 2, 3, 4, 8
- [56] Z. Teed and J. Deng. Raft: Recurrent all-pairs field transforms for optical flow. In *European conference on computer vision*, pages 402–419. Springer, 2020. 6
- [57] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri. Learning spatiotemporal features with 3d convolutional networks. In *Proceedings of the IEEE international conference on computer vision*, pages 4489–4497, 2015. 6
- [58] S. Tulyakov, M.-Y. Liu, X. Yang, and J. Kautz. Mocogan: Decomposing motion and content for video generation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1526–1535, 2018. 2
- [59] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017. 4
- [60] R. Villegas, J. Yang, S. Hong, X. Lin, and H. Lee. Decomposing motion and content for natural video sequence prediction. *arXiv preprint arXiv:1706.08033*, 2017. 2
- [61] R. Villegas, D. Erhan, H. Lee, et al. Hierarchical long-term video prediction without supervision. In *International Conference on Machine Learning*, pages 6038–6046. PMLR, 2018.
- [62] R. Villegas, A. Pathak, H. Kannan, D. Erhan, Q. V. Le, and H. Lee. High fidelity video prediction with large stochastic recurrent neural networks. *arXiv preprint arXiv:1911.01655*, 2019. 2
- [63] C. Vondrick, H. Pirsiavash, and A. Torralba. Generating videos with scene dynamics. *arXiv preprint arXiv:1609.02612*, 2016. 2
- [64] T.-C. Wang, M.-Y. Liu, J.-Y. Zhu, G. Liu, A. Tao, J. Kautz, and B. Catanzaro. Video-to-video synthesis. *arXiv preprint arXiv:1808.06601*, 2018. 2, 7
- [65] T.-C. Wang, M.-Y. Liu, A. Tao, G. Liu, J. Kautz, and B. Catanzaro. Few-shot video-to-video synthesis. *arXiv preprint arXiv:1910.12713*, 2019.
- [66] Y. Wang, P. Bilinski, F. Bremond, and A. Dantcheva. Imaginator: Conditional spatio-temporal gan for video generation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1160–1169, 2020. 2, 7
- [67] Y. Wang, F. Bremond, and A. Dantcheva. Inmodegan: Interpretable motion decomposition generative adversarial network for video generation. *arXiv preprint arXiv:2101.03049*, 2021. 2
- [68] L.-Y. Wei and M. Levoy. Fast texture synthesis using tree-structured vector quantization. In *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, pages 479–488, 2000. 3
- [69] L.-Y. Wei, S. Lefebvre, V. Kwatra, and G. Turk. State of the art in example-based texture synthesis. In *Eurographics 2009, State of the Art Report, EG-STAR*, pages 93–117. Eurographics Association, 2009. 3
- [70] Y. Wexler, E. Shechtman, and M. Irani. Space-time video completion. In *Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2004. CVPR 2004.*, volume 1, pages I–I. IEEE, 2004. 3, 4
- [71] L. Wolf, M. Guttman, and D. Cohen-Or. Non-homogeneous content-driven video-retargeting. In *Proceedings of the Eleventh IEEE International Conference on Computer Vision (ICCV)*, 2007. 2, 8
- [72] Z. Yang, W. Zhu, W. Wu, C. Qian, Q. Zhou, B. Zhou, and C. C. Loy. Transmomo: Invariance-driven unsupervised video motion retargeting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5306–5315, 2020. 2, 7

A. Implementation Details

All RGB values in the videos are scaled to $[-1, 1]$. All runs were conducted on Quadro RTX 8000 GPU.

Creating the Spatio-Temporal Pyramid Given a downscaling factor r (where $r = (r_H, r_W, r_T)$) and a minimal size m_S, m_T , we keep downscaling the input video in all dimensions until it “hits” the minimal size in the spatial or temporal dimensions. Assume we hit the minimal spatial dimension first, we keep on downscaling the temporal dimension until reaching its minimal size, while keeping the spatial dimensions fixed on its minimal size (and the opposite goes if we first hit the temporal dimension, keeping on downscaling the spatial dimensions while keeping the temporal fixed). The minimal size of the spatial dimensions, m_S , is the minimum between both height and width (namely, no spatial dimension will be smaller than m_S).

We use cubic downscaling interpolation for both temporal and spatial dimensions. We tried to use nearest interpolation on the temporal dimension, because it might make more sense sometimes, but found that in most applications it performed the same or worse.

Diverse Generation We used Gaussian noise with standard deviation of 2 – 5. Downscaling factor is 0.82 for the spatial dimensions (height and width) and 0.87 for the temporal dimension. The minimal size of the pyramid is set to 3 frames with minimal spatial dimension of 15 pixels. We use patch size $(3 \times 7 \times 7)$, where 3 is in the temporal dimension. We use 5 EM-like iterations in each scale of the pyramid. When the number of voxels ($T \times H \times W$) is larger than 3,000,000 we change the number of EM-like iterations to 1, and the patch-size to $(3 \times 5 \times 5)$. This change reduce runtime without hurting the quality of the results.

Video Analogies For all examples we use patch size $(3 \times 5 \times 5)$ and $\alpha = 1$ (for completeness score). For all-pairs examples we use downscaling factor is 0.9 for all dimensions. The minimal size of the pyramid is set to 3 frames with minimal spatial dimension of 20 pixels. 1 EM-like iterations per scale. For sketch-to-video examples we use downscaling factor of 0.78 for all dimensions and minimal size is 5 frames and minimal spatial dimension of 35. 3 EM-like iterations per scale (and 1 for the last two scales, to save runtime). Runtime per result is about 1 minute.

Layout-Appearance Tradeoff in Video Analogies Since we are trying to create a new video whose spatio-temporal layout (modeled with the dynamic structure) taken from one video and its appearance from another, there’s an inherent tradeoff of which of the two we want to be better preserved in the result. The dynamic structure is “enforced” by using

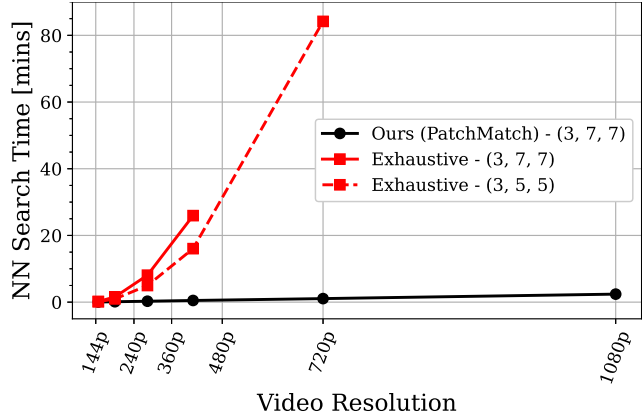


Figure 9. **Nearest Neighbor Search Comparison** using PatchMatch (in our method) vs. exhaustive search (used by GPNN [24]). GPNN exceeds GPU memory at medium resolution (480p) with the original patch size $(3, 7, 7)$. The dashed line with smaller patch size $(3, 5, 5)$ is intended to show the quadratic trend with more data points.

the auxiliary channels in Q and K . Removing these channels would generate a video that is much more similar in its appearance to S but bears less resemblance to the spatio-temporal layout of C . We can control this tradeoff by setting an upper limit in the pyramid from which we stop using the auxiliary channels. In our results it was best to set the maximal scale at half the pyramid height.

B. PatchMatch Implementation Details

In all applications we use $L2$ as the distance function between. In Fig. 9 we show another more detailed comparison between our PatchMatch implementation and the exhaustive nearest-neighbor search used by GPNN. Our implementation has time complexity of $O(n \times d)$ and $O(n)$ additional memory (where n is the video size and d is the patch size), compared to GPNN, with time complexity of $O(n^2 \times d)$ and memory footprint $O(n \times d)$. This is easily seen in the figure. We use the same propagation and random search steps as in the original PatchMatch paper [9], using the “jump flood” scheme [45]. In each PatchMatch iteration we look at 4 neighbors at distance $step$ (with additional small noise for the exact position of the neighbor, and without) and a random search. However, we only use 15 PatchMatch iterations per VPNN usage, this is done by searching for $step = 8, 4, 1$ 5 times.

C. Comparison details

For each input video in HP-VAE-GAN and SinGAN-GIF datasets we generated the same number of random sample as publicly available (10 generations for each video in HP-VAE-GAN dataset, and 6 generations for each video in

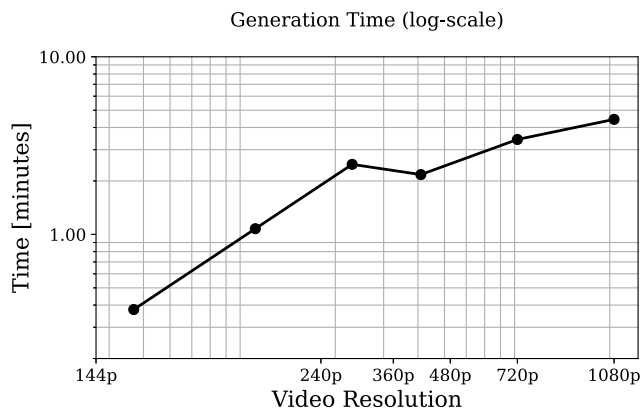


Figure 10. **Generation Time** Note y-axis is in log-scale. This is the same graph as in Fig. 6, shown here in log-scale so it would be easier to get a sense of the actual runtime needed to generate a random 13-frames video in the relevant resolution (x-axis).

SinGAN-GIF dataset), and compared their SVFID and diversity.

Video Diversity Index The video adaptation of the *diversity* index (originally proposed for images by [52]) is: given an input video, the standard deviation of each video position (3D RGB element in the video, converted to grayscale) is calculated across all generated samples, and then averaged across all pixels. This is then divided by the standard deviation of the voxels in the input video.